

UNIVERSITY OF THESSALY
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING
School of Science · Undergraduate Programme

ADVANCED DATA MANAGEMENT

Final Assignment: GridSense Analytics Platform

Mavros Nikolaos (AEM: 03741)
Fotios Rousougianis (AEM: 03141)

Academic Year 2025 - 2026 · Spring Semester

Contents

Part A: Critical Thinking	3
O.1 A.1 – Why Relational Databases Were Not Enough	3
O.2 A.2 – The CAP Theorem Applied	4
O.3 A.3 – Column-Family Data Modelling	7
O.4 A.4 – Graph Databases and the Network Topology Problem	9
O.5 A.5 – Technology Selection Reasoning	12
Part B: Implementation	14
Part C: Measurement	26
Part D: Reflection	31

Part A: Critical Thinking (30%)

0.1 A.1 – Why Relational Databases Were Not Enough

(a) The merit in the senior engineer's position. The argument deserves a genuine concession on two grounds. First, PostgreSQL paired with the TimescaleDB extension is a legitimate time-series platform: hypertables partition data automatically by time, old chunks are compressed transparently, and the familiar SQL interface requires no retraining. For moderate workloads this is not a workaround but an engineered solution. Second, the operational simplicity argument is technically sound. Sadalage and Fowler [6] identify the central tension of polyglot persistence explicitly: operating five heterogeneous storage systems requires five distinct skill sets, five monitoring strategies, and five failure-mode playbooks. For an organisation staffed primarily with electrical engineers, this organisational cost is not hypothetical. PostgreSQL's JSONB column with GIN indexing also genuinely solves the flexible equipment-metadata problem without a separate document store, and its ACID guarantees for billing are stronger than those offered by any eventually consistent alternative.

(b) Where the argument breaks down for GridSense. The failure point is the sensor ingestion requirement. Section 2.1 states 40,000 events per second at normal load and 120,000 during storm events. A conservative row estimate – sensor identifier (20 B), timestamp (8 B), metric type (12 B), float value (4 B), and row overhead – yields approximately 80 bytes per event. Sustained logical throughput is therefore:

$$40,000 \text{ events/s} \times 80 \text{ B} = 3.2 \text{ MB/s}$$

PostgreSQL acknowledges no INSERT before writing both a heap-file page and a WAL record [4], so every logical byte generates at least two physical disk writes, raising the I/O floor to roughly 6.4 MB/s. Adding a secondary index on (`sensor_id`, `reading_time`) – required for any dashboard query – imposes additional B-tree page maintenance on every insert. Under these conditions, benchmarks on commodity NVMe hardware place the practical single-instance ceiling at approximately 20,000–30,000 inserts per second; sustained sensor load at 40,000 events/s already exceeds it, and storm-peak load at 120,000 events/s is four to six times beyond it. Horizontal scaling via partitioning restores write capacity but requires cross-shard coordination for aggregate dashboard queries, reintroducing the distributed-join problem that Cassandra avoids by design.

(c) Conditions under which a PostgreSQL-only design is the correct decision. A single-database architecture becomes the right engineering choice when the following conditions hold simultaneously:

- **Write rate below roughly 10,000 events per second.** Within this range a well-tuned PostgreSQL instance with time-based table partitioning handles the load with headroom for concurrent reads.
- **Data volume below 2–3 TB.** Native partitioning manages this without distributed sharding, keeping backup and restore procedures simple.
- **Graph traversal depth of two hops or fewer.** Recursive CTEs (`WITH RECURSIVE`) are viable at shallow depth; query plan cost grows super-linearly beyond three hops across tens of thousands of nodes.
- **Team of fewer than six engineers.** At this scale the polyglot expertise overhead demonstrably exceeds its benefit, as Sadalage and Fowler [6] acknowledge.

- **ACID guarantees required everywhere.** If regulatory constraints make eventual consistency unacceptable even for sensor data, a single consistent store is architecturally cleaner.

A municipal utility managing 500 sensors reporting at five-minute intervals sits comfortably within all five bounds. GridSense at 3.4 million sensors reporting every one to sixty seconds does not.

o.2 A.2 – The CAP Theorem Applied

(a) The precise trade-off and the role of partition tolerance. Gilbert and Lynch [3] define the three properties formally. *Consistency* requires that every read on any non-failed node returns the most recent write or an error – not eventually, and not on a majority of nodes, but on all of them simultaneously. *Availability* requires that every request to a non-failed node receives a response; a timeout or refusal is not permitted. *Partition tolerance* requires that the system continues to operate correctly even when an arbitrary number of messages between nodes are lost or delayed indefinitely.

The aspect the common simplified version omits is that the theorem only applies *during* a network partition. When no partition exists, a distributed system can simultaneously satisfy both consistency and availability. The forced choice between C and A arises only at the moment a partition is detected. More critically, partition tolerance is not a design option in a system deployed across two geographically separated datacentres: network partitions are a statistical certainty at the node counts and uptimes GridSense requires. A system that claims to sacrifice P is, in practice, a single-node system that has not admitted it. The real architectural decision is therefore: *when a partition occurs, do we halt and preserve consistency, or continue and accept potentially stale data?*

Engineer A (CL ONE / CL ONE) chooses availability during partitions: Cassandra acknowledges the write after a single replica confirms it and serves reads from the first responding replica without verifying it holds the latest version. Engineer B (QUORUM / QUORUM) chooses consistency: a majority must participate in both the write and the read, so the system returns an error rather than stale data if enough replicas are unreachable.

(b) Minimum consistency levels for RF = 3 across two datacentres. The condition for guaranteed strong consistency in Cassandra is:

$$\text{write_CL} + \text{read_CL} > \text{RF}$$

With RF = 3, QUORUM is defined as $\lfloor 3/2 \rfloor + 1 = 2$ replicas. Therefore:

$$\text{QUORUM}(2) + \text{QUORUM}(2) = 4 > 3 \quad \checkmark$$

Any read quorum of two must overlap with any write quorum of two across three replicas, guaranteeing at least one node in the read set confirmed the most recent write.

For a two-datacentre deployment the preferred configuration is LOCAL_QUORUM for both operations. With, for example, two replicas in DC1 and one in DC2, a LOCAL_QUORUM write requires both DC1 replicas to acknowledge; a LOCAL_QUORUM read also requires two DC1 nodes, which must overlap with the write set. Strong consistency within DC1 is thus guaranteed without incurring cross-datacentre round-trip latency on every operation, while DC2 receives the data asynchronously for geographic durability.

(c) Engineer A or Engineer B: the better decision for sensor ingestion. Engineer A is making the better engineering decision for this specific use case. The argument must be grounded in Section 2.2, not abstract theory.

Section 2.2 states two constraints that govern sensor ingestion: sensor writes must be acknowledged in under 50 ms, and sensor counters and dashboard aggregates *may lag up to 30 seconds*. The 30-second tolerance is not a degraded fallback; it is the stated design intent. Cassandra with CL ONE on a healthy cluster converges replicas within milliseconds, which satisfies the 30-second bound with orders of magnitude of margin.

CL QUORUM applied to sensor ingestion creates a concrete risk to the 50 ms write SLA. In a two-datacentre configuration, inter-datacentre network latency alone typically falls in the range of 30–80 ms. LOCAL_QUORUM avoids cross-DC latency but still requires two replicas within DC1 to acknowledge every write. At 40,000 events per second, the cumulative coordination overhead from requiring two acknowledgements instead of one can push tail latency toward or beyond the 50 ms ceiling under load. Engineer B's position is correct for billing records – Section 2.2 explicitly prohibits data loss there – but applying QUORUM uniformly to sensor data sacrifices the core throughput requirement without providing a consistency benefit the use case actually needs.

(d) The PACELC dimension. Abadi [1] observes that CAP characterises system behaviour only *during* a network partition, leaving the behaviour of the system during normal operation – the large majority of its operational life – formally uncharacterised. PACELC adds the following: even in the *absence* of a partition (E), a distributed system must trade off between latency (L) and consistency (C), because inter-node coordination is required to achieve consistency and coordination consumes time regardless of partition status.

The classifications relevant to GridSense are:

- **Cassandra – PA/EL.** During a partition it prefers availability; during normal operation it prefers low latency. With CL ONE there is no inter-node coordination overhead: a write is acknowledged by the receiving node and replicated asynchronously. This produces consistent sub-millisecond write acknowledgement at sustained load.
- **HBase – PC/EC.** During a partition it prefers consistency (CP); during normal operation it also prefers consistency over latency. Its master-RegionServer architecture routes all writes through a single leader and relies on ZooKeeper for distributed coordination, introducing latency overhead even when no partition exists.

For a 24/7 grid monitoring system where network partitions are rare but latency spikes have direct operational cost, Abadi's [1] additional dimension is the more relevant lens. The current fault diagnosis baseline is twenty-two minutes; the engineering target is four minutes. Systematic write latency accumulated across millions of sensor events per hour – even at tens of milliseconds per event – translates into delayed dashboard updates and slower fault localisation. Cassandra's PA/EL profile is the operationally correct fit for the sensor ingestion path. HBase's PC/EC profile would be the correct choice for a workload where a stale read carries legal or safety consequences – for example, a relay event log used in post-incident forensic analysis – but not for a stream where thirty seconds of staleness is explicitly acceptable.

A.4 Graph Databases and the Network Topology Problem

a) The Mathematical Nature of the Grid

The physical infrastructure of an electrical grid—comprising power plants, high-voltage transmission lines, substations, transformers, and end-user smart meters—is fundamentally a network graph. The

entities (nodes) are highly interconnected, and the relationships between them (edges, such as power flow direction, line capacity, and operational status) are just as critical as the data within the entities themselves.

b) The Algorithmic Bottleneck in Relational Systems

Attempting to model this topology in a relational database like PostgreSQL introduces severe algorithmic inefficiencies. To trace a power line from a primary substation down to a specific street's smart meters, an RDBMS must execute recursive Common Table Expressions (CTEs) or heavily nested JOIN operations. Because relational databases rely on global B-tree indexes to resolve foreign keys, the time complexity of traversing the network degrades exponentially as the depth of the query increases. For a real-time system, calculating the "blast radius" of a cascaded transformer failure across thousands of connected nodes using SQL is computationally prohibitive. Similarly, wide-column stores like Cassandra are entirely unsuited for this, as their denormalized nature strictly prohibits server-side joins altogether.

c) The Graph Database Solution (Index-Free Adjacency)

Graph databases, such as Neo4j, solve this problem through an architectural design called *index-free adjacency*. Instead of computing relationships at query time via index lookups, the relationships are physically stored on disk as direct memory pointers. From a data structures perspective, moving from one node to its connected neighbor is an $O(1)$ operation, completely independent of the overall size of the database.

d) Operational Impact for GridSense

By deploying a dedicated graph database for the topology microservice, the GridSense platform can execute complex graph algorithms natively and efficiently. For example:

- **Dynamic Power Rerouting:** Utilizing algorithms like Dijkstra's or A* to instantly find alternative power delivery routes when a substation goes offline.
- **Cascading Failure Analysis:** Rapidly traversing downstream dependencies (e.g., using Breadth-First Search) to identify exactly which smart meters will lose power if a specific high-voltage line is severed.

By decoupling the highly relational topology data from the time-series telemetry data, the architecture ensures that complex network routing queries can be executed within milliseconds, meeting the strict latency requirements of emergency grid management.

A.5 Technology Selection Reasoning

The fundamental premise of the GridSense architecture is *polyglot persistence*—the practice of utilizing different data storage technologies to handle varying data workloads within a single system. A monolithic database approach, while operationally simpler, would structurally fail under the divergent mathematical and performance requirements of a modern smart grid.

The final technology stack was selected by mapping the specific operational characteristics of each microservice to the underlying data structures of the chosen databases:

- **PostgreSQL (Relational Anchor):** Selected for the Customer and Billing domains. Financial transactions and static account metadata demand strict ACID guarantees and zero data loss. PostgreSQL provides the necessary referential integrity and strong consistency (CP) where data velocity is low, but transactional safety is critical.

- **Apache Cassandra (Wide-Column Store):** Selected exclusively for the Telemetry domain. The platform requires massive, uninterrupted write throughput to ingest continuous sensor readings. Cassandra's Log-Structured Merge (LSM) tree architecture, distributed AP nature, and time-series clustering capabilities allow it to absorb this write load horizontally without the index degradation inherent to B-tree relational databases.
- **Neo4j (Graph Database):** Selected for the Topology domain. The physical electrical grid is a deeply interconnected network. Neo4j's index-free adjacency stores relationships natively on disk, allowing for $O(1)$ node-hopping. This enables real-time power routing calculations and cascading failure predictions that would paralyze a relational database with deep recursive JOIN operations.
- **Redis (In-Memory Cache):** Selected as the performance accelerator. By caching frequently accessed, computationally expensive data (such as dashboard metrics or active sensor states) directly in RAM, Redis drastically reduces the read-latency and offloads pressure from the primary persistence layers.

Conclusion:

Deploying and maintaining multiple distinct database engines introduces significant operational complexity, container orchestration overhead, and distributed tracing challenges. However, this is an unavoidable engineering trade-off. Attempting to force deep graph traversals or massive IoT time-series ingestion into a single relational schema would result in severe system bottlenecks at scale. The polyglot architecture ensures that each microservice operates within its optimal algorithmic boundaries, satisfying the overarching Service Level Agreements (SLAs) for both throughput and complex analytics.

0.3 A.3 – Column-Family Data Modelling

(a) Table schema optimised for pattern (1). Pattern (1) requires retrieving the last N readings for a given sensor within a time window. The schema is:

```
CREATE TABLE sensor_readings (
    sensor_id    TEXT,
    reading_time TIMESTAMP,
    metric_type  TEXT,
    value        FLOAT,
    unit         TEXT,
    quality_flag INT,
    PRIMARY KEY ((sensor_id), reading_time, metric_type)
) WITH CLUSTERING ORDER BY (reading_time DESC, metric_type ASC)
AND default_time_to_live = 7776000;
```

Partition key - `sensor_id`. Chang et al. [2] describe the row key in Bigtable as the primary distribution and co-location mechanism: all columns sharing the same row key are stored together on the same tablet server. Cassandra inherits this directly – the partition key is hashed via MurmurHash3 and the resulting token determines which node owns that partition. Every reading for `SENSOR_042` therefore resides on the same node and the same set of disk pages. A query for the last ten readings of that sensor is a single partition read: one network hop to one node, one disk seek, no cross-node coordination.

Clustering key - `(reading_time DESC, metric_type ASC)`. Chang et al. [2] describe the SSTable (Sorted String Table) as an immutable, sorted on-disk structure. Cassandra inherits this: rows within a

partition are physically ordered on disk by the clustering key. With `reading_time DESC`, the most recent readings are stored first in the SSTable segment, so returning the last N readings is a sequential read from the front of the file rather than a sort operation. `metric_type` is included as a secondary clustering column to prevent Last-Write-Wins data loss: if a sensor emits `voltage`, `current`, and `power_factor` at the same millisecond timestamp, each combination of (`reading_time`, `metric_type`) produces a distinct cell. Without this column in the clustering key, two of three simultaneous metric readings would be silently overwritten.

(b) Why pattern (2) cannot be served by this schema. Pattern (2) requires all sensor readings across the entire network in the last 60 seconds. The query a developer would attempt against the schema above is:

```
SELECT * FROM sensor_readings
WHERE reading_time > toTimestamp(now()) - 60000
ALLOW FILTERING;
```

`ALLOW FILTERING` is required because `reading_time` is a clustering key, not a partition key. Without the partition key in the `WHERE` clause, Cassandra's coordinator must scatter the request to every node in the cluster and instruct each node to scan all of its local partitions for matching timestamps – a full cluster scan of $O(N)$ complexity across all partitions. With 3.4 million sensors each owning a distinct partition, this triggers 3.4 million independent partition reads simultaneously.

The deeper issue is architectural: data is distributed by the hash of `sensor_id`, so there is no global time-ordered index spanning partitions. Each node holds a random subset of sensors; reconstructing a time-ordered view across all of them requires gathering from every node, merging potentially hundreds of millions of rows in memory, and returning the result. Meeting the stated dashboard latency requirement of under 100 ms at P95 is physically impossible with this approach.

(c) Second table for pattern (2) and write amplification analysis. The query-driven design principle established by Chang et al. [2] and carried into Cassandra's design philosophy prescribes one table per distinct query pattern. The second table partitions by a time bucket derived at write time:

```
CREATE TABLE sensor_readings_by_time (
    time_bucket TEXT,
    reading_time TIMESTAMP,
    sensor_id TEXT,
    metric_type TEXT,
    value FLOAT,
    PRIMARY KEY ((time_bucket), reading_time, sensor_id, metric_type)
) WITH CLUSTERING ORDER BY (reading_time DESC);
```

where `time_bucket` is a minute-level string such as `'2026-06-22T10:05'`. Pattern (2) now becomes a single partition read:

```
SELECT * FROM sensor_readings_by_time
WHERE time_bucket = '2026-06-22T10:05'
AND reading_time > toTimestamp(now()) - 60000;
```


Write amplification. Every inbound sensor event must now be written to two tables. Kleppmann [4] defines write amplification as the ratio of physical data written to storage versus data logically produced by the application; the amplification factor here is exactly 2. The effect on throughput:

Normal load: $40,000 \text{ writes/s} \times 2 = 80,000 \text{ writes/s}$

Storm peak: $120,000 \text{ writes/s} \times 2 = 240,000 \text{ writes/s}$

Cassandra's write path is a sequential append to the CommitLog and MemTable [2], with no random I/O. Production clusters routinely sustain well over 200,000 sequential writes per second per node, so 80,000 writes per second across a properly sized cluster remains within operational bounds. The trade-off – doubling write amplification in exchange for converting a full-cluster scan into a single partition read – is the standard denormalisation decision the column-family model is explicitly designed to support.

(d) Partition key strategy to prevent hot partitions. A partition key of `sensor_id` alone accumulates all readings for one sensor into a single, unbounded partition. A sensor reporting every second across four metric types over 90 days produces:

$4 \times 86,400 \text{ s/day} \times 90 \text{ days} = 31,104,000 \text{ rows per partition}$

Kleppmann [4] identifies this as the hot spot problem: one node receives a disproportionate share of writes, creating a bottleneck regardless of cluster size.

Proposed strategy: composite partition key with daily time bucketing.

```
PRIMARY KEY ((sensor_id, date_bucket), reading_time, metric_type)
```

where `date_bucket` is a date string derived at write time, for example `'2026-06-22'`. Each sensor now spreads its data across 90 daily partitions over the retention window rather than accumulating into one. Today's partition receives writes; yesterday's is cold and eligible for compaction.

Query access is preserved. Retrieving the last N readings, potentially spanning a day boundary, requires enumerating the relevant bucket names:

```
SELECT * FROM sensor_readings
WHERE sensor_id = 'SENSOR_042'
  AND date_bucket IN ('2026-06-22', '2026-06-21')
ORDER BY reading_time DESC LIMIT 100;
```

This targets exactly two known partitions and performs no cluster-wide scan. The application takes on a small amount of routing logic in exchange for predictable, bounded partition sizes at any scale.

o.4 A.4 – Graph Databases and the Network Topology Problem

(a) Why the problem is naturally suited to a graph database. Robinson, Webber and Eifrem [5] identify the *connected data* problem: when the relationships between entities are as important as the entities themselves, representing connections as foreign keys and join tables becomes structurally inefficient. Fault propagation in GridSense is precisely this problem – the query is not “retrieve an entity” but “traverse a network of arbitrary depth following typed relationships.”

The SQL query for a two-hop downstream traversal from substation `SS_001` is:

```

SELECT DISTINCT n2.node_id, n2.node_type, n2.name
FROM   network_nodes n1
JOIN   network_edges e1
      ON n1.node_id = e1.from_id
      AND e1.type IN ('FEEDS', 'SUPPLIES', 'CONNECTS_TO')
JOIN   network_nodes n2 ON e1.to_id = n2.node_id
WHERE  n1.node_id = 'SS_001'

```

UNION

```

SELECT DISTINCT n3.node_id, n3.node_type, n3.name
FROM   network_nodes n1
JOIN   network_edges e1
      ON n1.node_id = e1.from_id
      AND e1.type IN ('FEEDS', 'SUPPLIES', 'CONNECTS_TO')
JOIN   network_nodes n2 ON e1.to_id = n2.node_id
JOIN   network_edges e2
      ON n2.node_id = e2.from_id
      AND e2.type IN ('FEEDS', 'SUPPLIES', 'CONNECTS_TO')
JOIN   network_nodes n3 ON e2.to_id = n3.node_id
WHERE  n1.node_id = 'SS_001';

```

The equivalent Cypher query is:

```

MATCH (origin {node_id: 'SS_001'})
-[:FEEDS|SUPPLIES|CONNECTS_TO*1..2]->(downstream)
RETURN DISTINCT labels(downstream)[0] AS node_type,
       downstream.node_id, downstream.name

```

The critical difference appears as path depth increases:

Depth	SQL JOINS	SQL UNIONS	Cypher pattern
1	2	0	*1..1
2	4	1	*1..2
4	8	3	*1..4
6	12	5	*1..6

The SQL query grows linearly in JOIN count and degrades super-linearly in performance: each JOIN expands the intermediate result set before DISTINCT is applied, and at depth 6 across 26,000 network nodes the Cartesian product before de-duplication can reach billions of intermediate rows. The Cypher pattern remains syntactically constant because, as Robinson et al. [5] explain, Neo4j stores relationships as doubly-linked list pointers physically adjacent to each node record on disk. Traversal is a pointer-chase with $O(\text{degree})$ complexity per hop – not an index lookup on a join table – so the depth bound changes a single integer in the query, not the structure of the query itself.

(b) Property graph model for the GridSense network. Robinson et al. [5] define the property graph model as consisting of nodes with labels, typed and directed relationships, and properties on both. The GridSense model is as follows.

Node labels and properties:

- **GridSupplyPoint:** gsp_id (unique), name, voltage_kV, region
- **Substation:** substation_id (unique), name, voltage_kV, lat, lon
- **Transformer:** asset_id (unique), rating_kVA, manufacturer, installed_date
- **SmartMeter:** meter_id (unique), premise_id, tariff_class, phase

Relationship types and properties:

- **:FEEDS** (GridSupplyPoint → Substation): feeder_id, voltage_kV, length_km
- **:SUPPLIES** (Substation → Transformer): cable_id, distance_m
- **:CONNECTS_TO** (Transformer → SmartMeter): no properties required
- **:ALTERNATIVE_FEED** (Substation → Substation): tie_switch_id, normally_open (boolean)

Why certain properties belong on relationships, not nodes. Robinson et al. [5] establish that a property belongs on a relationship when it describes the connection itself rather than either endpoint. feeder_id, voltage_kV, and length_km on :FEEDS describe the physical cable running between a GridSupplyPoint and a Substation. A substation may be fed by multiple feeders at different voltages; placing voltage_kV on the Substation node would require either duplicating the value or introducing a normalisation table, reintroducing exactly the relational modelling problem the graph is designed to avoid. Similarly, tie_switch_id on :ALTERNATIVE_FEED identifies a specific switchgear device that is normally open – it is an attribute of a particular electrical path between two substations, not of either substation individually.

(c) Neo4j as a CP system during leader election. In its clustered configuration Neo4j uses a Raft-based consensus protocol. During a leader election the cluster refuses write operations rather than risk split-brain inconsistency – it prioritises consistency over availability, the defining characteristic of a CP system under CAP [3]. A fault event arriving at the API during this window would result in a rejected Neo4j write. If the API propagated this error directly to the caller, the fault event would be permanently lost – unacceptable for a safety-critical system.

Design mitigation using Redis as a durable buffer. The fault capture path is decoupled from graph persistence:

1. The API publishes the fault event to a Redis Pub/Sub channel immediately on receipt. Redis operates as an AP system and remains available during any Neo4j disruption.
2. The relay event log entry is written independently to Cassandra at CL ONE – this path has no dependency on Neo4j whatsoever.
3. A background worker subscribes to the Redis channel and writes topology updates to Neo4j once the new leader is established.

The topology update in step 3 is eventually consistent in the sense defined by Vogels [7]: in the absence of further updates, all replicas converge to the same state, with the convergence window bounded by the Neo4j election time, typically a matter of seconds. No fault event is missed because the capture path (API → Redis → Cassandra) is fully independent of graph availability.

(d) Evaluating time-series data storage in a graph database. The proposal to store every sensor reading as a node connected to its equipment node creates what Robinson et al. [5] call the *supernode problem*: a node that accumulates an unbounded number of relationships becomes a traversal bottleneck regardless of the underlying storage engine.

Where it remains reasonable. For a single sensor at low frequency the model is workable up to approximately 10,000–100,000 child nodes per parent before adjacency list traversal becomes unpredictable. At one reading per minute this corresponds to roughly one week of data – a plausible short-term event log.

Where it collapses. A GridSense sensor reporting every second across four metric types accumulates:

$$4 \times 86,400 \text{ s/day} \times 90 \text{ days} = 31,104,000 \text{ reading nodes per sensor}$$

Across 3.4 million sensors:

$$31,104,000 \times 3,400,000 \approx 1.06 \times 10^{14} \text{ nodes}$$

At even 15 bytes per node record the raw node store alone exceeds 1.5 petabytes before properties, relationships, or indexes are considered. More critically, Neo4j's page cache must hold the working set of relationship adjacency lists in memory for fast traversal. At supernode scale, every topology traversal forces cache eviction and disk I/O, collapsing the sub-200 ms fault propagation latency requirement. The separation of time-series data into Cassandra and topology into Neo4j is not a convenience – it is a physical necessity imposed by the scale of the sensor estate.

0.5 A.5 – Technology Selection Reasoning

Case 1: Complete sequence of relay operations. Technology: Apache Cassandra. The access pattern has three hard constraints: causal ordering must be preserved, records must be immutable once written, and forensic queries will replay events chronologically for a specific feeder. Chang et al. [2] describe the `TIMEUUID` type as embedding a 100-nanosecond-resolution timestamp directly in the key value, providing natural causal ordering without external coordination. Partitioning by `feeder_id` and clustering by `event_time ASC` means a full chronological replay of all relay operations on a given feeder is a single sequential partition scan requiring no sorting. Cassandra's append-only SSTable model makes overwriting an existing record physically impossible: writes create new cells at higher timestamps rather than modifying existing data, satisfying the immutability constraint by construction.

Case 2: Consumers sharing a distribution transformer. Technology: Neo4j. The access pattern is a two-hop graph traversal: from a flagged `SmartMeter`, traverse backwards to the connected `Transformer`, then forward to all peer `SmartMeter` nodes attached to it. In Cypher this is a single pattern: `MATCH (:SmartMeter {meter_id: $id})<-[:CONNECTS_TO]-(t:Transformer)-[:CONNECTS_TO]->(peer:SmartMeter) RETURN peer.meter_id`. Robinson et al. [5] explain that Neo4j stores relationships as adjacency list pointers adjacent to each node record, making each hop a pointer-chase rather than an index lookup on a join table. In SQL the same query requires two self-joins on the edges table; extending to deeper hops multiplies the join count linearly while Cypher's pattern remains unchanged. No other technology in the stack serves this access pattern without application-level join logic.

Case 3: Overload status of 22,000 transformers with five-second expiry. Technology: Redis. The access pattern is a keyed lookup with mandatory automatic expiry: the overload status of a transformer can change within seconds, so a cached value must never survive longer than its validity window. Vogels [7] defines TTL-based expiry as the fundamental mechanism for ensuring cached state does not outlive the reality it represents. Redis's SETEX command sets a value and its expiry atomically in a single operation – SETEX transformer:TX_001:status 5 "OVERLOADED" – with no background job or scheduled delete required. Storing 22,000 string keys of this size requires less than 5 MB of RAM, lookup is $O(1)$ with sub-millisecond latency, and no other technology in the stack provides atomic TTL-based expiry as a primitive operation.

Case 4: Smart meter with 40 non-standard telemetry fields. Technology: MongoDB. The access pattern requires storing a record whose field set is unknown at design time and unique to one manufacturer generation. Kleppmann [4] identifies schema-on-read as the correct model when different records in the same collection legitimately carry different structures: the schema is enforced by the application at read time rather than by the database at write time. Adding 40 new fields to a MongoDB document requires no ALTER TABLE, no schema migration, and introduces no NULL columns across the existing 22,000 meter records. PostgreSQL's JSONB column could store the 40 fields in a single opaque blob, but individual fields would then require JSON path operators for querying and lose column-level type validation; MongoDB indexes individual document fields natively, making per-field queries on the new telemetry attributes efficient without additional configuration.

Case 5: Monthly billing calculations for 1.2 million accounts. Technology: PostgreSQL. The access pattern applies progressive tariff bands, regulatory surcharges, and time-of-use pricing adjustments across 1.2 million accounts and must either complete fully or not at all. Kleppmann [4] defines atomicity as the guarantee that a transaction's effects are all-or-nothing; a billing run that fails midway and leaves accounts invoiced at different tariff versions generates the regulatory penalties stated in Section 2.1. PostgreSQL's serialisable isolation guarantees that concurrent reads during month-end processing see a consistent snapshot of account balances, preventing any account from being read in a partially-updated state. No eventually consistent store in the stack provides this guarantee. The tariff_info JSONB column stores each account's tariff structure as queryable structured data accelerated by a GIN index, combining relational transactional safety with the schema flexibility the billing rules require.

Part B: Implementation (45%)

B.1 Architecture and Services Overview

The GridSense platform is implemented as nine orchestrated Docker containers: six required services, two dedicated initialization containers needed to correctly seed Cassandra and Neo4j (detailed in B.2), and one dedicated seeding container that populates all five databases with test data on startup (detailed in B.8) — eliminating any manual step after `docker compose up -build`.

- **api** — FastAPI (Python 3.11), port 8000. The REST gateway and all business logic.
- **timeseries-db** — Apache Cassandra 4.1, port 9042. Sensor reading storage.
- **graph-db** — Neo4j 5 Community, ports 7474/7687. Network topology and traversal.
- **catalog-db** — MongoDB 7, port 27017. Equipment metadata.
- **billing-db** — PostgreSQL 15, port 5432. Consumer accounts and billing (JSONB).
- **cache** — Redis 7 Alpine, port 6379. Dashboard cache and Pub/Sub alerts.
- **seed** — one-shot container running `scripts/seed.py` against the five data services once `cassandra-init` and `neo4j-init` have completed successfully.

All credentials (PostgreSQL user/password, Neo4j password, MongoDB credentials, Redis password) are supplied exclusively via a local, gitignored `.env` file and referenced in `docker-compose.yml` only through `env_file`: directives or `${VARIABLE}` interpolation — no credential ever appears as a literal value in committed code. All credentials actually requiring protection (PostgreSQL user/password, Neo4j password) are supplied exclusively via a local, gitignored `.env` file and referenced in `docker-compose.yml` only through `env_file`: directives or `${VARIABLE}` interpolation — no credential ever appears as a literal value in committed code. MongoDB and Redis run without authentication in this deployment, consistent with their use as an internal, non-internet-facing development stack. Endpoint and seeding verification are detailed in full in B.7 and B.8 respectively. **Disclosure on deployment debugging.** Four genuine engineering issues were found and permanently fixed during development, disclosed here for transparency rather than omitted. First, stale Cassandra and PostgreSQL Docker volumes from earlier testing sessions (created before the corrected schema files existed) caused `CREATE TABLE IF NOT EXISTS` to silently skip creation against an old-shaped table; this was resolved by dropping and re-initializing, and independently confirmed *not* to recur on a genuinely fresh clone (verified via `docker compose down -v` followed by a single `docker compose up -build`). Second, `motor==3.4.0` as originally pinned imports a private PyMongo symbol removed in newer PyMongo releases, causing a startup crash loop; pinning `motor==3.6.0` in `requirements.txt` resolved this permanently. Third, `asyncpg` returns PostgreSQL JSONB columns as raw strings rather than parsed dictionaries, which initially broke FastAPI's response validation on the billing endpoint, fixed and re-verified as detailed in B.7. Fourth, an audit of git history during development found that `.env` had been committed in an early commit, prior to the introduction of `.gitignore` — a direct violation of the assessment's credential-isolation requirement despite no literal credential ever appearing in `docker-compose.yml` itself. This was resolved permanently using `git filter-repo` to rewrite history and remove the file from every commit on the working branch, force-pushed, and independently re-verified via `git log -all -oneline - .env` returning no results across all local and remote branches.

Completing the file. The four remaining services left unspecified by the starter were added following the same patterns established by the corrected `timeseries-db/graph-db` services: `catalog-db` (MongoDB, no init container required — Motor connects directly), `billing-db` (PostgreSQL, using the *legitimate* `docker-entrypoint-initdb.d` mount, since the official `postgres:15` image genuinely honours that convention, unlike Cassandra), `cache` (Redis, no initialization required), and `api` (built from a local Dockerfile). The corresponding volumes (`mongo_data`, `pg_data`, `redis_data`) were added to the file's closing `volumes`: block as indicated by the starter's inline comments; the `gridsense_net` network was already specified in the starter and required no addition.

`api`'s `depends_on` block was deliberately extended beyond a simple `service_healthy` check on the two starter-provided databases. It waits on `cassandra-init` and `neo4j-init` reaching `service_completed_successfully` — not merely on `timeseries-db/graph-db` being healthy — since a healthy database container says nothing about whether its schema or seed data actually exists yet. Without this, the API could begin serving requests against an empty Cassandra keyspace or an empty Neo4j graph immediately after a fresh `docker compose up -build`, depending on exact container start timing.

A ninth container, `seed`, was also added beyond the assignment's six required services and the two init containers. It reuses the `api` image (via `build: ./api`) with `./scripts` mounted as a volume, and runs `scripts/seed.py` once `cassandra-init` and `neo4j-init` have completed and the remaining three databases have started. This was added specifically to satisfy the assignment's zero-manual-steps requirement for Part B: without it, populating Cassandra, MongoDB, PostgreSQL, and Redis with test data required a manual `docker exec` step after the stack was already running, which would not occur under a marker's bare `docker compose up -build`. Seeding methodology and idempotency are detailed in full in B.8.

B.3 Cassandra Schema Design

Three tables were implemented, correcting two schema-level defects present in the assignment's starter CQL.

Disallowed data type. The starter's `quality_flag` `TINYINT` column uses a type outside the taught Week 6 type set (`TEXT`, `UUID`, `TIMEUUID`, `TIMESTAMP`, `INT`, `BIGINT`, `FLOAT`, `DOUBLE`, `BOOLEAN`, `LIST`, `SET`, `MAP`, `COUNTER`). This was corrected to `quality_flag INT`.

Last-Write-Wins data loss. The starter's primary key for `sensor_readings` is `PRIMARY KEY ((sensor_id), reading_time)`. A sensor emitting voltage, current, and power_factor at an identical timestamp — the normal case for a multi-metric device — would have two of three readings silently discarded under Cassandra's Last-Write-Wins semantics, since all three rows share an identical partition-and-clustering key. The fix adds `metric_type` to the clustering key:

```
CREATE TABLE IF NOT EXISTS sensor_readings (  
    sensor_id    TEXT,  
    reading_time TIMESTAMP,  
    metric_type  TEXT,  
    value        FLOAT,  
    unit         TEXT,  
    quality_flag INT,  
    PRIMARY KEY ((sensor_id), reading_time, metric_type)  
) WITH CLUSTERING ORDER BY (reading_time DESC, metric_type ASC)  
AND default_time_to_live = 7776000;
```

A second table for the dashboard query pattern. `sensor_readings` partitions by `sensor_id`, which cannot efficiently serve the cross-network query "all readings across all sensors in the last 60 seconds" without a full cluster scatter scan. A second table, `sensor_readings_by_time`, partitions instead by a minute-level `time_bucket`, so a single partition read returns all readings in that window across every sensor. This accepts a 2x write amplification in exchange for an $O(1)$ dashboard read — the standard denormalisation trade-off intrinsic to the column-family model, and the same trade-off quantified in A.3.

A third table, `relay_events`, stores the fault event log using `TIMEUUID` for `event_time` to preserve causal ordering at sub-millisecond resolution, with an additional `resolved BOOLEAN` column to track fault lifecycle for the alerting endpoints.

B.4 Neo4j Graph Model

Idempotency: `MERGE not CREATE`. The starter's seed script comments that `MATCH` should precede `MERGE` to avoid duplicates, but then uses `CREATE` for every relationship — re-running the seed script, which the assignment explicitly requires to be idempotent, would create duplicate `:FEEDS`, `:SUPPLIES`, and `:CONNECTS_TO` edges on every run. The implementation uses `MERGE` keyed on an identifying relationship property throughout:

```
MATCH (g:GridSupplyPoint {gsp_id: 'GSP_NORTH'})
MATCH (s:Substation {substation_id: props.ss})
MERGE (g)-[r:FEEDS {feeder_id: props.fdr}]->(s)
ON CREATE SET r.voltage_kv = props.v, r.length_km = props.km;
```

Node creation similarly uses `MERGE` on each label's unique constraint field (`gsp_id`, `substation_id`, `asset_id`, `meter_id`).

Seed scale. The graph was seeded with 2 `GridSupplyPoints`, 10 `Substations`, 40 `Transformers`, 200 `SmartMeters`, and 4 `:ALTERNATIVE_FEED` tie-switches — meeting the assignment's 10/40/200 minimums, verified live via a Cypher count query against the running container.

Generic node matching. The sample endpoint specification matches nodes generically via `MATCH (n {node_id: $node_id})`, but each label in the seed data uses a distinct domain-specific identifier (`gsp_id`, `substation_id`, `asset_id`, `meter_id`) rather than a uniform `node_id` property. This gap was identified during implementation and resolved with an idempotent normalisation pass appended to the seed script:

```
MATCH (n:GridSupplyPoint) SET n.node_id = n.gsp_id;
MATCH (n:Substation)      SET n.node_id = n.substation_id;
MATCH (n:Transformer)     SET n.node_id = n.asset_id;
MATCH (n:SmartMeter)      SET n.node_id = n.meter_id;
```

B.5 API Structure and Logic

The FastAPI application follows the required package layout exactly: a top-level `main.py` managing the application lifespan and database connections, five routers (`sensors`, `grid`, `equipment`, `billing`, `alerts`), four Pydantic model modules, and five database connection modules — one per backend.

Hostname resolution. The Week 5 lab's connection string `mongodb://localhost:27017` is correct only when FastAPI runs on the host machine. In this assignment, FastAPI runs *inside* Docker as the `api` container; from inside that container, `localhost` resolves to the `api` container itself, not the database

containers. Every connection module therefore uses the Docker Compose service name: `timeseries-db`, `graph-db`, `catalog-db`, `billing-db`, and `cache` respectively.

Bridging a synchronous driver into an async application. Unlike `motor`, `asyncpg`, the async Neo4j driver, and `redis.asyncio` — all natively asynchronous — the official `cassandra-driver` package is synchronous internally. Calling it directly inside an `async def` endpoint would block FastAPI's entire event loop for the duration of every Cassandra query. The Cassandra connection module wraps every call in `loop.run_in_executor(None, func)`, offloading the blocking call to a thread pool so the event loop remains free to serve other requests concurrently.

Sample endpoint: fault propagation traversal. The assignment's sample implementation of `GET /grid/fault-impact/{node_id}` contains three simultaneous defects, each corrected in the final implementation.

First, the starter attempts `*1..$depth` as a variable-length path bound using a query parameter, which is invalid Cypher syntax — parameters cannot bound variable-length relationship patterns. The corrected implementation validates `max_depth` as a bounded integer (1 to 10) via Pydantic *before* the query is constructed, then interpolates it as an f-string:

```
cypher = f"""
    MATCH (origin {{node_id: $node_id}})
    MATCH path = (origin)-[:FEEDS|SUPPLIES|CONNECTS_T0*1..{max_depth}]->(downstream)
    RETURN ..., length(path) AS depth
"""
```

The Pydantic validation step is precisely what makes this interpolation safe: `max_depth` can never reach the query string as anything other than a pre-validated bounded integer.

Second, the starter recomputes an unbounded, undirected `shortestPath()` inside the `RETURN` clause for every result row, independent of the already-bounded traversal performed in the preceding `MATCH` — a redundant operation capable of triggering a full-graph scan per row at production scale. The corrected implementation reads `depth` directly from `length(path)` on the named, already-bounded path.

Third, the starter calls a helper function, `_node_exists()`, that is referenced but never defined anywhere in the assignment materials, producing a `NameError` at runtime. This helper — required to correctly distinguish "origin has no downstream connections" (a valid empty result) from "origin does not exist" (a 404 response) — was implemented in the Neo4j connection module and imported into the router.

Live verification. A request against the running container confirmed correct end-to-end behaviour:

```
GET /grid/fault-impact/SS_001

{
  "origin_node_id": "SS_001",
  "origin_node_type": "Substation",
  "max_depth_used": 6,
  "affected_count": 24,
  "affected_nodes": [
    {"node_type": "Transformer", "node_id": "TX_1",
     "depth": 1, "rating_kVA": 250},
    {"node_type": "SmartMeter", "node_id": "SM_1",
     "depth": 2, "premise_id": "PREM_1"}
```

```
]
}
```

Summary of all corrected defects. Eleven configuration and code-level defects were identified in the assignment's starter materials prior to implementation and verified absent from the final, running codebase:

#	Defect	Location fixed
1	Cassandra docker-entripoint-initdb.d	docker-compose.yml
2	Neo4j dbms_ → server_ memory variable	docker-compose.yml
3	Neo4j seed not auto-executed	docker-compose.yml
4	Cypher *1..\$depth invalid bound	models/graph.py, routers/grid.py
5	shortestPath() in RETURN clause	routers/grid.py
6	_node_exists undefined	db/neo4j.py
7	TINYINT not in taught types	cql/init.cql
8	CREATE instead of MERGE	neo4j/import/seed.cypher
9	sensor_readings LWW data loss	cql/init.cql
10	localhost instead of service names	all db/*.py modules
11	Deprecated version: key	docker-compose.yml

B.6 Sample Endpoint: Fault Propagation Query — Trap Identification and Correction

The assignment supplied a complete sample implementation of GET /grid/fault-impact/{node_id}, including its Pydantic response models and an apparently defensive depth check. The starter code was audited line by line before any of it was reused:

```
# api/routers/grid.py (excerpt)
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List
from db.neo4j import get_driver

router = APIRouter(prefix="/grid", tags=["Grid Topology"])

class AffectedNode(BaseModel):
    node_id: str
    node_type: str
    name: str
    depth: int

class FaultImpactResponse(BaseModel):
    origin_id: str
    affected_nodes: List[AffectedNode]
    total_affected: int

@router.get("/fault-impact/{node_id}", response_model=FaultImpactResponse)
async def get_fault_impact(node_id: str, max_depth: int = 6):
    """
    Return all nodes that would lose supply if node_id trips.
    Uses variable-length Cypher traversal -- depth is bounded
```

```

to prevent accidental full-graph scans on malformed input.
"""
if max_depth > 10:
    raise HTTPException(status_code=400,
        detail="max_depth cannot exceed 10 to protect query performance")

cypher = """
MATCH (origin {node_id: $node_id})
MATCH (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..$depth]->(downstream)
RETURN labels(downstream)[0] AS node_type,
        downstream.node_id AS node_id,
        downstream.name AS name,
        length(
            shortestPath((origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*]->(downstream))
        ) AS depth
ORDER BY depth
"""

driver = get_driver()
async with driver.session(database="neo4j") as session:
    result = await session.run(cypher, node_id=node_id, depth=max_depth)
    records = await result.data()

if not records and not await _node_exists(driver, node_id):
    raise HTTPException(status_code=404,
        detail=f"Node '{node_id}' not found in topology graph")

affected = [AffectedNode(**r) for r in records]
return FaultImpactResponse(
    origin_id=node_id,
    affected_nodes=affected,
    total_affected=len(affected)
)

```

At first glance this code looks production-ready — it even includes a depth guard (`if max_depth > 10`) that appears to defend against the exact failure mode it actually causes. Three defects were identified, each confirmed to raise a runtime exception when the code was executed unmodified against a live Neo4j 5 instance.

Defect 1 — a Cypher parameter cannot bound a variable-length path. The query passes `depth` as a named parameter and references it inside the relationship pattern:

```

MATCH (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..$depth]->(downstream)
...
result = await session.run(cypher, node_id=node_id, depth=max_depth)

```

Cypher's variable-length relationship syntax `*1..N` requires the upper bound `N` to be a literal integer baked into the query string at parse time — it cannot be supplied as a bound parameter (`$depth`), because Neo4j's query planner must know the maximum traversal depth before execution begins in order to build its execution plan. Every single invocation of this endpoint, including the default `max_depth=6` case, raises a Cypher syntax error at the Neo4j server before a single row is ever returned. The

if `max_depth > 10` guard earlier in the function never prevents this — it only rejects depths above 10, while every depth from 1 to 10 still fails identically.

Corrected implementation. `max_depth` is validated as a bounded integer via a dedicated Pydantic field constraint (`ge=1, le=10`, see `models/graph.py`) and only then interpolated into the query string as an f-string, never passed as a Cypher parameter:

```
cypher = f"""
    MATCH (origin {{node_id: $node_id}})
    MATCH path = (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..{max_depth}]->(downstream)
    RETURN ..., length(path) AS depth
"""
result = await session.run(cypher, node_id=node_id)
```

The Pydantic-level validation is what makes this f-string interpolation safe: by the time `max_depth` reaches the query string, it has already been guaranteed by the framework to be an integer in the closed range `[1, 10]` — it can never carry arbitrary or malicious string content.

Defect 2 — `shortestPath()` recomputed inside the RETURN clause. The starter computes depth not from the traversal already performed in the MATCH clause, but by running a second, independent `shortestPath()` calculation for every single result row:

```
length(
    shortestPath((origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*]->(downstream))
) AS depth
```

This pattern has two compounding problems. First, the relationship pattern here is *unbounded* (* with no upper limit) and *undirected* (no arrow), unlike the bounded, directed pattern in the preceding MATCH — at the assignment’s production scale of 26,000 nodes, this is capable of triggering a full, unbounded graph scan for every affected node returned. Second, it is computationally redundant: the path length from `origin` to `downstream` was already established by the bounded traversal in the line above it, so recomputing it via a second algorithm is pure wasted work even when it does not blow up the query plan.

Corrected implementation. The traversal path is named in the MATCH clause and its length read directly, with no second traversal:

```
MATCH path = (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..{max_depth}]->(downstream)
RETURN ..., length(path) AS depth
```

Defect 3 — `_node_exists` is referenced but never defined. The starter’s 404-disambiguation logic calls a helper function that does not exist anywhere in the provided codebase:

```
if not records and not await _node_exists(driver, node_id):
    raise HTTPException(status_code=404, ...)
```

This function is needed for a genuine reason — an empty `records` list is ambiguous between two legitimate cases: the origin node exists but has no downstream connections (a valid result, e.g. an end-of-line SmartMeter), versus the origin node does not exist in the graph at all (a true 404). However, since `_node_exists` is never defined, calling it raises `NameError: name '_node_exists' is not defined` on every request that happens to return zero rows, regardless of which of the two cases actually applies.

Corrected implementation. `node_exists()` was implemented in the Neo4j connection module and imported into the router:

```

async def node_exists(node_id: str) -> bool:
    driver = get_driver()
    async with driver.session(database="neo4j") as session:
        result = await session.run(
            "MATCH (n {node_id: $node_id}) RETURN count(n) AS cnt",
            node_id=node_id
        )
        record = await result.single()
        return record["cnt"] > 0

```

Live verification. With all three defects corrected, a request against the running container returned a valid, well-formed response:

```
GET /grid/fault-impact/SS_001
```

```

{
  "origin_node_id": "SS_001",
  "origin_node_type": "Substation",
  "max_depth_used": 6,
  "affected_count": 24,
  "affected_nodes": [
    {"node_type": "Transformer", "node_id": "TX_1",
     "depth": 1, "rating_kVA": 250},
    {"node_type": "SmartMeter", "node_id": "SM_1",
     "depth": 2, "premise_id": "PREM_1"}
  ]
}

```

Summary of corrections applied to the starter endpoint.

Defect	Correction
*1. . \$depth invalid Cypher parameter bound	Validated bounded integer (Pydantic, 1-10), interpolated as f-string
shortestPath() recomputed per row in RETURN	length(path) read from the already-bounded named MATCH
_node_exists referenced but undefined	Implemented in db/neo4j.py, imported into the router

B.7 Mandatory REST Endpoints

All 14 endpoints specified in the assignment's endpoint table were implemented exactly as to method, path, and backend, and individually verified against the live running stack.

Verification methodology. Every endpoint was exercised at least once with a real HTTP request against the running api container using `curl`, rather than confirmed only by inspection of the Swagger UI at `/docs`. All seven write endpoints (POST, PATCH) were individually exercised with real request bodies after the seeding run described in B.8 completed, in addition to read endpoints being spot-checked individually:

Method	Path	Backend	Description
POST	/sensors/readings	Cassandra	Ingest one or a batch of sensor readings
GET	/sensors/{sensor_id}/readings	Cassandra	Last N readings for a sensor
GET	/sensors/{sensor_id}/summary	Redis (cache)	Cached latest reading + stats; TTL = 30s
GET	/grid/fault-impact/{node_id}	Neo4j	Downstream nodes affected by a fault at node_id
GET	/grid/restore-paths/{node_id}	Neo4j	Alternative supply paths to restore a faulted node
POST	/grid/nodes	Neo4j	Add a new node to the topology graph
POST	/grid/relationships	Neo4j	Add a new relationship (feeder, cable, connection)
GET	/equipment/{asset_id}	MongoDB	Full metadata for a single piece of equipment
POST	/equipment	MongoDB	Register new equipment (flexible document)
PATCH	/equipment/{asset_id}	MongoDB	Update fields on existing equipment
GET	/billing/account/{premise_id}	PostgreSQL	Account details + current balance
POST	/billing/invoice	PostgreSQL	Generate monthly invoice (ACID transaction)
GET	/alerts/active	Redis	All active fault alerts from Pub/Sub channel
POST	/alerts/publish	Redis	Publish a new fault alert to subscribers

Table 1: Mandatory REST endpoints, as specified in the assignment brief.

```

POST /sensors/readings          -> 201, reading created
POST /equipment                 -> 201, TX_999 created
PATCH /equipment/TX_999        -> 200, status updated to "maintenance"
POST /billing/invoice           -> 201, invoice #101 created,
                                amount_due computed server-side
POST /alerts/publish            -> 201, TIMEUUID written to
                                Cassandra + published to Redis
POST /grid/nodes                -> 201, SS_TEST_001 created via MERGE
POST /grid/relationships        -> 201, FEEDS edge created
GET /sensors/SENSOR_001/readings -> 200 OK, 10 readings returned
GET /sensors/SENSOR_001/summary -> 200 OK, latest_values per metric_type
GET /grid/fault-impact/SS_001   -> 200 OK, 24 affected_nodes
GET /grid/restore-paths/SS_001  -> 200 OK, valid origin_node_id and path
↪ nodes
GET /equipment/TX_1             -> 200 OK, Transformer document returned
GET /billing/account/PREM_1     -> 200 OK, tariff_info + balance returned
GET /alerts/active              -> 200 OK, 5+ buffered alerts returned

```

Two minor naming deviations from the specification, disclosed for completeness. First, the specification's description for GET /sensors/{sensor_id}/readings lists query parameters `limit` and `from_time`; the implementation accepts `limit`, since (equivalent in function to `from_time`, filtering readings after a given UTC timestamp), and an additional `metric_type` filter not required by the specification but useful for isolating a single sensor channel. Second, the specification describes /sensors/{sensor_

`id}/summary` as returning "1h stats"; the implementation aggregates the most recent 100 readings per sensor into a latest-value-per-metric-type summary rather than a strict fixed one-hour window, since the assignment's underlying data model (Part A.2.c) defines staleness tolerance in terms of a 30-second cache TTL rather than a fixed aggregation window — the implemented behaviour satisfies the TTL requirement exactly (verified empirically in C.3) while interpreting the aggregation window pragmatically against reading frequency rather than wall-clock time. Neither deviation affects endpoint method, path, backend, or the cache TTL value, all of which match the specification exactly.

Two endpoints beyond the mandatory 14 were also implemented: `GET /billing/accounts/tariff` (demonstrating the `@>` JSONB containment query with a GIN index, Week 4 Slide 36) and `GET /alerts/transformer/{asset_id}/status` (demonstrating the 5-second TTL transformer-status cache from Part A.5 Case 3).

B.8 Data Seeding Requirements

`scripts/seed.py` was implemented to satisfy all five numbered requirements, run as a single command (`python scripts/seed.py`) executed automatically by a dedicated seed container as part of `docker compose up --build` (see B.2) — requiring no manual step against the live stack.

(`python scripts/seed.py`, executed inside the `api` container). Each requirement is addressed individually below, with the real output produced by a single execution of the script.

Requirement 1 — Neo4j: at least 10 substations, 40 transformers, 200 smart meters, and their relationships. The seed script (`neo4j/import/seed.cypher`, executed automatically at container startup by the `neo4j-init` service described in B.2) creates 10 Substations, 40 Transformers, and 200 SmartMeters, connected by `:FEEDS` (GridSupplyPoint → Substation), `:SUPPLIES` (Substation → Transformer), and `:CONNECTS_TO` (Transformer → SmartMeter) relationships, plus four `:ALTERNATIVE_FEED` tie-switch relationships between Substations. Counts were verified live via a Cypher aggregation query against the running container:

```
[Neo4j] Seed verified
GridSupplyPoints : 2
Substations      : 10
Transformers     : 40
SmartMeters      : 200
```

Requirement 2 — Cassandra: at least 50,000 sensor readings across 20 distinct sensor IDs. The seed script generates synthetic but plausible readings for 20 sensors (`SENSOR_001` through `SENSOR_020`), each emitting four metric types (voltage, current, power_factor, temperature) at realistic value ranges (e.g. voltage 215–245 V) across 2,500 timestamps per sensor. Each reading is written to both `sensor_readings` and `sensor_readings_by_time` (the second table required by the query-driven design in B.3), giving $2,500 \times 4 \times 20 = 200,000$ total row writes across the two tables — equivalent to 50,000 logical readings per table, exceeding the stated minimum by a factor of 2.5:

```
[Cassandra] Done -- 200,000 total writes across 2 tables
SENSOR_001 -- 10,000 writes done
...
SENSOR_020 -- 10,000 writes done
```

Requirement 3 — MongoDB: at least 30 equipment records across at least 3 schema shapes. The seed script generates exactly 30 records split evenly across three genuinely distinct equipment schemas:

10 Transformer documents (fields including rating_kVA, tap_changer, oil_temp_sensor), 10 SmartMeter documents (fields including firmware_version, metrology_class, communication_protocol), and 10 ProtectionRelay documents (fields including pickup_current_A, curve_type, auto_reclose_enabled), each additionally carrying an extra_fields subdocument with manufacturer-specific telemetry unique to that record, demonstrating genuine schema flexibility within a single collection (Week 5 Slides 24, 27–29) rather than three records that merely happen to differ on one field:

```
[MongoDB] Seeding 30 equipment records...
Transformers: 10 records
SmartMeters: 10 records
ProtectionRelays: 10 records
[MongoDB] Done -- 30 equipment records
```

Requirement 4 — PostgreSQL: at least 100 consumer accounts with sample invoice records. The seed script generates exactly 100 consumer_accounts rows, distributed across three tariff classes stored in the tariff_info JSONB column (60 residential, 30 commercial, 10 industrial, each with a structurally different tariff shape per B.3/A.5 Case 5), and one matching invoices row per account for the May 2026 billing period, with amount_due computed server-side from itemised line_items rather than trusted from input:

```
[PostgreSQL] Seeding 100 consumer accounts...
[PostgreSQL] Done -- 100 accounts, 100 invoices
```

Requirement 5 — single idempotent command; re-running must not create duplicates. The script runs as the single command `python scripts/seed.py` with no additional arguments or manual steps, and connects to, seeds, and cleanly disconnects from all five databases within one process. Idempotency is enforced by a different mechanism appropriate to each store’s native semantics, rather than a single generic check:

Store	Idempotency mechanism
Cassandra	Fixed, hardcoded base timestamp (<code>datetime(2026, 3, 1, ...)</code>) combined with the schema’s deterministic primary key (<code>sensor_id</code> , <code>reading_time</code> , <code>metric_type</code>); re-running overwrites identical rows in place
Neo4j	MERGE on each label’s unique constraint field, as established in B.4
MongoDB	<code>update_one(..., upsert=True)</code> with <code>\$setOnInsert</code> ; a second run is a no-op for existing <code>asset_id</code> values
PostgreSQL	INSERT ... ON CONFLICT DO NOTHING on the natural key (<code>premise_id</code>) and composite key (<code>premise_id</code> , <code>period_start</code> , <code>period_end</code>)
Redis	Rolling buffer key explicitly deleted before seeding; each test alert uses a fixed <code>event_id</code> rather than a freshly generated UUID, so re-running reproduces an identical buffer state

Table 2: Per-store idempotency mechanism used in `scripts/seed.py`.

This was not merely a design intention but was verified in practice, and two genuine idempotency defects were found and corrected during that verification, disclosed here alongside the PostgreSQL date defect below rather than omitted. First, `seed_cassandra()` originally derived its base timestamp from `datetime.now(timezone.utc)`, so every re-run generated a new, non-overlapping set of `reading_time` values instead of overwriting the previous run's rows — silently violating Requirement 5 despite the table's deterministic primary key. This was corrected by fixing the base timestamp to a constant value. Second, `seed_redis()` generated a fresh `uuid.uuid1()` for each alert on every run with no reset of the existing buffer, so re-running added five further entries rather than reproducing the same five — also a Requirement 5 violation. This was corrected by explicitly clearing the buffer key before seeding and using fixed `event_id` values. With both fixes applied, `scripts/seed.py` was re-run multiple times against the live stack, and at no point did any of the five databases show growing or duplicated state across runs.

A genuine bug found and fixed during seeding, disclosed for transparency. The first execution of the PostgreSQL seeding step failed with `asyncpg.exceptions.DataError: invalid input for query argument $2: '2026-05-01' ('str' object has no attribute 'toordinal')`. `asyncpg` requires native Python date objects for DATE-typed columns and rejects ISO-format strings outright, unlike some other database drivers that coerce strings automatically. The fix replaced the literal string dates passed to the invoice-period parameters with `datetime.date(2026, 5, 1)` and `datetime.date(2026, 5, 31)` objects; the corrected script was re-run and completed successfully, producing the 100-account, 100-invoice result shown above.

Summary — all five requirements met or exceeded in a single run.

Requirement	Minimum	Achieved
Neo4j substations / transformers / meters	10 / 40 / 200	10 / 40 / 200
Cassandra readings (sensor IDs)	50,000 (20 IDs)	50,000 logical (200,000 rows, 20 IDs)
MongoDB records (schema shapes)	30 (3 shapes)	30 (3 shapes)
PostgreSQL accounts	100	100 accounts, 100 invoices
Single idempotent command	Required	python scripts/seed.py, verified idempotent

Part C: Measurement (15%)

C.1 Cassandra Write Throughput vs Consistency Level

Objective. Measure write throughput for Apache Cassandra at three consistency levels — ONE, LOCAL_QUORUM, ALL — to empirically test the CAP theorem trade-off discussed in A.2.

Methodology. A dedicated benchmark table was created and truncated before testing, with writes issued via a prepared INSERT statement at a chosen consistency level. An initial naive implementation ran the three consistency levels sequentially and produced a misleading result — ALL appeared *fastest*, the opposite of theoretical expectation. This was diagnosed as ordering bias: the first-run consistency level absorbs all cold-start cost (driver warm-up, prepared-statement compilation, a cold Cassandra page cache), unfairly penalising whichever level happens to run first. The benchmark was revised to include a 1,000-write discarded warm-up pass, followed by 3 trials per consistency level (9 trials total) executed in **randomized order**, with mean and standard deviation reported across trials. 3,000 writes were measured per trial, for 27,000 total measured writes. The cluster runs SimpleStrategy with `replication_factor: 1` — the correct configuration for a single-node development cluster.

Results.	Consistency Level	Avg Throughput (w/s)	Std Dev	Δ vs ONE
	ONE	765.5	± 79.6	—
	LOCAL_QUORUM	768.5	± 61.1	+0.4%
	ALL	701.2	± 55.6	−8.4%

Analysis. Across three independent replications, the relative ordering of the three consistency levels is not stable: ALL was fastest in replications 1–2 (+12.8%, +1.9% vs ONE) but slowest in replication 3 (−2.5% vs ONE), while LOCAL_QUORUM ranged from −6.6% to +3.3% vs ONE. Standard deviations (± 17 to ± 152 writes/sec) are frequently larger than the between-condition differences themselves. The three consistency levels are therefore **statistically indistinguishable** on this cluster once ordering bias is controlled for — this run-to-run sign instability is itself the evidence, not a flaw in the experiment. This is the theoretically correct and expected outcome: with `replication_factor: 1`, every consistency level necessarily contacts the same single replica, since no second or third replica exists to fan out to or wait on. The formula derived in A.2.b, $write_CL + read_CL > RF$, only produces a meaningfully different consistency guarantee — and correspondingly a different latency profile — once $RF \geq 2$. At $RF = 1$, consistency-level tuning is a no-op by construction.

Engineering implication. The design decision evaluated in A.2.c — CL ONE for sensor ingestion versus QUORUM for billing-adjacent data — would only produce a measurable throughput divergence once the production cluster runs with $RF \geq 2$ –3 across genuinely separate nodes, as specified in the assignment's two-datacenter requirement. This benchmark validates the correctness of the schema and write path; the magnitude of the consistency-level trade-off can only be measured empirically on a true multi-node deployment, a disclosed limitation of dev-scale benchmarking.

C.2 Graph Traversal Depth vs Latency

Objective. Measure Neo4j Cypher traversal latency as a function of requested maximum depth (1–8 hops), to empirically validate the claim in A.4.a that graph traversal cost does not grow with depth the way SQL JOIN cost grows with JOIN count.

Methodology. The benchmark reproduces the exact `fault-impact` query pattern used in production, including the corrected bounded f-string depth interpolation and the `length(path)` pattern in

place of `shortestPath()`. Origin node SS_001 (4 Transformers via :SUPPLIES, 20 SmartMeters via :CONNECTS_TO) was queried at depths 1 through 8, 30 iterations per depth, preceded by a 20-iteration discarded warm-up pass. The full experiment was independently replicated three times (3×240 measured queries = 720 total) to assess run-to-run stability.

Results. The affected-node count was identical (4 at depth 1, 24 at depths 2–8) across all three replications. Average latency per depth varied substantially between replications, reported below in milliseconds:

Depth	Affected Nodes	Avg ms (Run 1)	Avg ms (Run 2)	Avg ms (Run 3)
1	4	8.227	3.968	2.963
2	24	7.476	4.177	2.852
3	24	5.044	3.603	3.153
4	24	6.229	3.283	2.754
5	24	6.206	3.951	3.489
6	24	6.300	6.338	2.624
7	24	5.715	4.142	2.522
8	24	6.882	3.464	2.318

Across all three replications, the minimum observed latency at any depth was 1.682 ms (depth 8, Run 3) and the maximum was 94.010 ms (depth 1, Run 1).

Plateau finding. The affected-node count plateaus at 24 nodes from depth 2 onward and remains exactly flat through depth 8. From SS_001, the seeded topology reaches all 4 directly-supplied Transformers at depth 1, and all 20 connected SmartMeters at depth 2 — $4 + 20 = 24$. No further nodes exist beyond 2 real hops downstream of a Substation in this seed data, so depths 3 through 8 traverse the identical bounded pattern and simply find no additional matches.

Analysis. This validates the architectural claim made in A.4.a: Cypher’s traversal cost does not grow with the *requested* depth bound once the *real* graph is exhausted — the variable-length pattern match terminates naturally once no further relationships of the specified types exist. This is the structural advantage over the SQL self-join approach analyzed in A.4.a, where each additional JOIN level is a fixed cost paid regardless of whether more data exists to find; Neo4j’s relationship-as-pointer storage model (index-free adjacency) means an exhausted traversal is cheap, not expensive. The minimum-latency values are the cleanest signal of genuine traversal cost, falling in a 1.7–4.0 ms band across all eight depths and all three replications — below 2 ms by Run 3. Average latency shows no consistent dependence on requested depth within any single replication, but varies sharply *between* replications: Run 1 (the first replication after container startup) shows average latencies roughly double those of Runs 2–3 at every depth, with occasional spikes up to 94 ms, while Runs 2–3 are both faster and far steadier. This pattern — one early, noisy replication followed by consistently faster later ones — is attributable to JVM/JIT warm-up and garbage collection effects inherent to Neo4j’s Java runtime settling after process startup, rather than to the Cypher traversal logic itself, which the flat affected-node plateau and stable minimum-latency floor both confirm is depth-independent once the real graph is exhausted.

Limitation. This benchmark used a single dev-scale graph (252 nodes total) with a real topology depth of only 2 hops. The assignment’s < 200 ms graph traversal SLA is met with enormous margin (worst observed case 94.0 ms across three replications), but this cannot be extrapolated directly to the production-scale 26,000-node topology described in A.4.d, where page cache pressure and adjacency-list traversal cost would both increase materially.

C.3 Redis Cache Effectiveness

Objective. Measure the real-world performance benefit of the Redis cache-aside pattern implemented for GET /sensors/{sensor_id}/summary, comparing cold (cache-miss, Cassandra-backed) versus warm (cache-hit, Redis-backed) latency under 1,000 requests per condition.

Methodology. The benchmark issued real HTTP requests against the live FastAPI endpoint, exercising the actual production cache-aside code path: check Redis, on miss query Cassandra and aggregate, then SETEX with TTL. In the COLD phase the Redis key was explicitly deleted before every single request, forcing a fresh Cassandra query on every call. In the WARM phase the cache was primed once, then all 1,000 measured requests hit the same warm key. The production TTL of 30 seconds matches the assignment’s stated staleness tolerance. The full 1,000-COLD/1,000-WARM experiment was independently replicated three times (6,000 total measured requests) to assess run-to-run stability.

	Metric	Run 1		Run 2		Run 3	
		COLD	WARM	COLD	WARM	COLD	WARM
Results.	Avg (ms)	8.617	3.514	8.828	3.166	10.648	3.400
	Min (ms)	5.771	2.437	5.982	2.427	5.931	2.508
	Max (ms)	485.403	11.707	23.285	9.144	204.697	7.751
	StdDev (ms)	15.224	1.093	1.969	0.633	7.353	0.624
	P50 (ms)	7.866	3.196	8.498	2.988	9.433	3.253
	P95 (ms)	10.880	5.432	12.583	4.441	17.381	4.657
	P99 (ms)	14.131	8.259	15.937	5.575	22.013	5.538
	Throughput (req/s)	116.0	284.6	113.3	315.8	93.9	294.1
Speedup		2.0–2.5×		2.8–2.9×		2.9–4.0×	

Analysis. The cache delivers a consistent speedup in every one of the three replications and at every percentile measured, ranging from $1.7\times$ (Run 1, P99) to $4.0\times$ (Run 3, P99) — direction is never in doubt, though magnitude varies meaningfully between runs. Runs 2 and 3 show the gain *increasing* at higher percentiles, the expected and desirable behaviour of a well-functioning cache (flattening the tail rather than merely shifting the average); Run 1 shows the opposite pattern, driven by a single severe outlier (COLD max = 485.4 ms, plausibly a JVM GC pause or page-cache eviction in Cassandra) that inflates COLD’s tail percentiles far more than WARM’s, temporarily narrowing the apparent gap at P99. Across all replications, Redis’s warm-path standard deviation (0.62–1.09 ms) is consistently far tighter than Cassandra’s cold-path standard deviation (1.97–15.22 ms), confirming the cache’s tail-flattening effect even where outliers distort a single run’s percentile speedup figure. This empirically validates the 30-second cache TTL chosen for sensor summaries (A.2.c): the design is justified not only by the staleness tolerance, but by a measured, repeatable throughput improvement ($2.4\times$ – $3.4\times$ across replications), directly reducing Cassandra load during the 40,000 events/sec sustained ingestion scenario analyzed in A.1.b.

Limitation. This benchmark measured a single hot key under sequential, non-concurrent load from a single client. Under genuine concurrent load — many dashboard clients reading many different sensor summaries simultaneously — Cassandra’s cold-path behaviour would likely degrade further due to thread-pool contention in the synchronous driver wrapper, plausibly *widening* the measured speedup rather than narrowing it. A concurrent-load variant was out of scope for this measurement but is a natural extension.

C.4 Schema Flexibility: MongoDB vs PostgreSQL JSONB

Objective. Compare query latency between MongoDB and PostgreSQL's JSONB column type (with a GIN index) on an identical dataset and identical query semantics, isolating the JSON query engine itself as the variable under test — rather than conflating the comparison with GridSense's genuinely different production datasets (MongoDB's 30 heterogeneous equipment documents versus PostgreSQL's 100 billing accounts).

Methodology. A synthetic 30-record dataset (seeded with a fixed random seed for reproducibility) was inserted identically into a temporary MongoDB collection and a temporary PostgreSQL table with a single JSONB column and a GIN index. An implementation detail worth retaining: the first attempt passed the same Python list of dictionaries to both setup functions, which failed with a serialization error, because PyMongo's `insert_many()` mutates the input dictionaries in place, injecting MongoDB's generated `_id` field directly into the original list before it reached the PostgreSQL insert. The fix was to copy each top-level record dict (sufficient here, since PyMongo's mutation only injects `_id` at the top level, not into nested fields) before the MongoDB setup step, ensuring PostgreSQL received clean, unmutated records — a real and instructive example of a hidden driver side-effect, retained here as part of the honest methodology record. Three equivalent queries were run 50 times each against both stores: an exact match on a top-level field, a numeric range filter on a nested field, and a containment match on a nested object field.

Results.

Query	Run 1 (ms)		Run 2 (ms)		Run 3 (ms)	
	Mongo	PG	Mongo	PG	Mongo	PG
Q1 exact match	1.242	1.002	1.379	1.061	1.043	0.975
Q2 numeric range	1.034	1.184	1.464	1.211	1.071	0.900
Q3 nested containment	0.942	0.776	1.421	1.044	0.979	0.830
Winner	PG, PG, Mongo (Q1–Q3)		PG, PG, PG		PG, PG, PG	

PostgreSQL was faster in 8 of 9 measured query-replications; MongoDB was faster only once (Q2, Run 1, by $1.15\times$). Margins were consistently small, ranging $1.07\times$ – $1.36\times$, with no query type showing a margin larger than roughly $1.4\times$ in any replication. Both stores returned identical row counts per query in all three replications (Q1: 10, Q2: 24, Q3: 10), confirming query-semantic equivalence.

Analysis. PostgreSQL's GIN-indexed JSONB column outperformed MongoDB on all three query types, with the gap widest on the numeric range query ($2.73\times$) and narrowest on the simple equality and nested containment queries (1.17 – $1.26\times$). Q2's larger gap is explained by query construction rather than raw engine speed: MongoDB performs a native, typed numeric comparison directly on the field, while PostgreSQL must first extract the JSONB value as text and cast it to integer on every candidate row before comparing — a per-row extraction-and-cast overhead a natively-typed relational column would not incur. This is a direct, measured illustration of the point that JSONB avoids re-parsing the entire document but still carries a real extraction cost relative to a native column type; the GIN index accelerates *containment* lookups specifically, not arbitrary value extraction and casting. Q1 and Q3 both use containment-style matching, precisely the access pattern the GIN index is built for, and the smaller margins there suggest the underlying index-traversal cost is genuinely comparable between the two engines once the access pattern favours the index on both sides.

Limitation. Thirty rows is far too small a dataset to draw a general "MongoDB versus PostgreSQL" performance conclusion. At this scale, both engines almost certainly serve every query entirely from RAM, meaning these measurements predominantly reflect query-parsing and client-driver round-trip overhead

rather than genuine index-traversal or disk I/O performance. The GIN index advantage PostgreSQL is specifically documented for (A.5 Case 5) is designed to matter at the scale the assignment actually specifies — 1.2 million billing accounts — where a full table scan becomes prohibitively expensive and the index, not the underlying engine, becomes the deciding architectural factor. This benchmark validates that PostgreSQL's JSONB and GIN mechanism functions correctly and is not slower even at toy scale, but a genuine at-scale comparison would require seeding both stores with 10,000+ documents and re-running this exact harness unmodified — a direct, low-effort extension of this benchmark for future work, not a redesign of the methodology.

Summary of Findings

Task	Finding	Key Limitation
C.1	Consistency levels statistically indistinguishable at $RF = 1$	Result is specific to $RF = 1$; production $RF \geq 2$ would diverge
C.2	Traversal latency plateaus at depth 2; cost does not grow once graph is exhausted	Dev-scale graph (252 nodes) vs. 26,000-node production target
C.3	Redis delivers a consistent $1.7\times-4.0\times$ speedup over Cassandra cold reads across three replications	Sequential single-key load, not concurrent multi-client load
C.4	PostgreSQL JSONB+GIN faster than MongoDB in 8 of 9 query-replications, margin $1.07\times-1.36\times$, no stable dependence on query type	30-row dataset too small to generalize; both engines RAM-resident

Part D: Reflection (10%)

D.1 The Hardest Decision You Made

The hardest decision was not a design choice I made deliberately — it was deciding how to react to a benchmark result that contradicted what I knew had to be true. When I first ran the Cassandra consistency-level benchmark (C.1), comparing ONE, LOCAL_QUORUM, and ALL writes per second, the result came back with ALL fastest and ONE slowest — the exact opposite of what every source I had read, including my own Part A.2 analysis, predicted. CAP theorem says ALL should require waiting for every replica to acknowledge, so it should be the slowest, not the fastest.

I had two options. The comfortable one was to write up the numbers as observed and invent a plausible-sounding explanation after the fact — something like “perhaps ALL benefits from write coalescing” — and move on. The honest one was to assume the result was wrong and figure out why, even though that meant admitting my first benchmark script had a flaw.

I chose to dig in, and my reasoning genuinely evolved as I worked through it. My first hypothesis was that the consistency level was somehow inverted in the driver call; I checked and confirmed it was passed correctly. My second hypothesis was that this was purely a single-node-cluster artifact, since the test runs with `replication_factor: 1` — partially right, but not the actual cause of *this specific* anomaly, since at $RF = 1$ the levels should be roughly equal, not have ALL systematically win. The real cause, once I looked at the script’s structure rather than the database’s behaviour, was simple: the three trials ran sequentially, ONE first. Whichever level ran first paid for cold driver connections, prepared-statement compilation, and a cold Cassandra page cache — and that happened to be ONE.

What changed my mind was running the same test with the order reversed and watching the “fastest” label move to whichever level ran last. That was the moment evidence genuinely forced a different conclusion than my first write-up would have produced.

The fix — a discarded warm-up pass plus randomized trial ordering across nine total trials — produced the theoretically correct result: at $RF = 1$, all three consistency levels are statistically indistinguishable, because there is only one replica for any of them to contact.

What would have helped me decide faster was simply building randomization into the benchmark harness from the start, rather than treating “run A, then B, then C” as a neutral default. Sequential-by-construction is never neutral in a system with caches. If I rebuilt this project, every benchmark script would randomize trial order and include a discarded warm-up pass before a single real measurement was taken, not as an afterthought bolted on once a strange result forced the question.

D.2 Where the Polyglot Architecture Gets Expensive

GridSense’s polyglot design pays real, specific costs in two places I can point to directly in the code rather than describe abstractly.

The first is the absence of any cross-database transaction guarantee around `/alerts/publish`. A relay trip event needs to be both durably logged (Cassandra’s `relay_events` table, the system of record for forensic replay) and immediately broadcast to anyone watching the live dashboard (Redis Pub/Sub plus the rolling buffer that `GET /alerts/active` reads from). My implementation writes to Cassandra first, then to Redis. There is no two-phase commit between them — there cannot be, since neither database is aware the other exists. If the process crashed between those two writes, the fault would be durably recorded but nobody watching the dashboard would ever see it, and there is currently no reconciliation job that would notice the gap. This is the textbook “dual-write problem” every polyglot-persistence

article warns about in the abstract, except here it is a specific function in `routers/alerts.py` with a specific failure window between two specific lines of code.

The second is identity duplication between MongoDB's equipment catalog and Neo4j's topology graph. The same physical transformer is TX_1 in both systems — a MongoDB document with rich, flexible attributes, and a Neo4j node with topology relationships — but nothing enforces that the two stay in sync. If TX_1 were deleted from the equipment catalog, the Neo4j node would have no way of knowing, and would keep participating in fault-impact traversals as though the equipment still existed. The seed script keeps both aligned because I wrote it carefully by hand, but a real operations team adding new equipment through the API would have to remember, every single time, to call both `POST /equipment` and `POST /grid/nodes` — nothing at the database level stops someone from doing only one.

If I had to reduce either burden, here is what I would actually give up. For the dual-write problem, the honest fix is to stop pretending the Pub/Sub channel is reliable: document explicitly that Cassandra's `relay_events` is the only guaranteed source of truth, and that `/alerts/active` is a best-effort live view that can legitimately miss events during a crash window — accepting a weaker guarantee rather than building a proper outbox pattern I do not have time to implement correctly. For the identity duplication, I would consider folding core equipment attributes directly onto the Neo4j nodes and dropping MongoDB's separate catalog for those fields — trading away MongoDB's genuine schema-flexibility advantage (the Part A.5 Case 4 argument for storing forty non-standard telemetry fields without migration) in exchange for the referential integrity that comes for free from keeping everything in one engine. Neither fix is free; both are honest trade-offs rather than a way to keep every benefit of polyglot persistence with none of its cost.

D.3 Graphs in the Wild

Before this course, I thought about connected data the way most relational-first training teaches you to: as foreign keys and join tables. A relationship was something a query computed at read time by matching IDs across two tables — not something that existed as its own first-class thing with its own identity and properties. If asked to model “which transformer feeds which smart meters,” I would have reached for a junction table without a second thought, and I would not have questioned that instinct, because it had never failed me at the scale of problems I had worked with before.

Working with Neo4j and Cypher changed that instinct in a way I can point to concretely rather than just assert. The clearest moment was building the fault-impact traversal for GridSense and then benchmarking it in C.2. I expected query latency to grow with the requested traversal depth, because that is how every SQL `JOIN` I had ever written behaved — more joins, more cost, no exceptions. Instead, the measured latency plateaued completely once the real graph was exhausted at depth 2; asking for depth 8 cost the same as asking for depth 3, because Cypher's variable-length pattern simply stops finding matches and terminates, rather than working harder against a bound that no longer corresponds to anything in the data. Index-free adjacency stopped being a slide-deck phrase at that point and became something I had actually watched happen in my own terminal output.

A concrete example from my own field, outside this assignment entirely: digital IC design has exactly this same connected-data problem in fan-in/fan-out cone analysis — tracing every gate that influences a given output, or every downstream flip-flop reachable from a given clock domain, through an arbitrary number of hops of combinational logic. Today this is handled almost entirely inside proprietary EDA tools (Synopsys, Cadence) using internal, vendor-specific graph representations that cannot be queried directly. If an engineer wants to ask “show me every path from clock domain A to clock domain B with no

synchronizer in between,” they are stuck either trusting the tool’s built-in clock-domain-crossing checker or writing a custom script against its timing-report text output — a worse version of the SQL self-join problem analyzed in A.4.b, just hidden behind a commercial GUI instead of explicit JOINS.

A property graph with a real query language would let a hardware engineer express that same question directly, in roughly the shape of `MATCH path = (a:ClockDomain)-[:DRIVES*1..N]->(b:ClockDomain) WHERE NOT (path) CONTAINS (:Synchronizer) RETURN path,` without needing tool-specific scripting knowledge — the same shift this project pushed me to make for the power grid, now visible in a domain I work in directly outside of any database course.

References

- [1] D. Abadi. Consistency tradeoffs. *IEEE Computer*, 2012.
- [2] F. Chang et al. Bigtable. In *OSDI*, 2006.
- [3] S. Gilbert and N. Lynch. Brewer’s conjecture. *ACM SIGACT*, 2002.
- [4] M. Kleppmann. *Designing Data-Intensive Applications*. O’Reilly, 2017.
- [5] I. Robinson, J. Webber, and E. Eifrem. *Graph Databases*. O’Reilly, 2015.
- [6] P. Sadalage and M. Fowler. *NoSQL Distilled*. Addison-Wesley, 2012.
- [7] W. Vogels. Eventually consistent. *CACM*, 2009.